

The unavoidable price of reliability is simplicity.

- C.A.R. Hoare

CSE 312

Operating Systems

Spring 2026

Synchronization

© 2013-2026 Yakup Genç

1

Today

- Synchronization

2

Timers

- Jiffies
 - Each timer tick, a variable called jiffies is incremented
 - It is thus (roughly) the number of HZ since system boot
 - A 32-bit counter incremented at 1000 Hz wraps around in about 50 days
 - We need 64 bits — but there's a problem

3

Potent and Evil Magic

- A 64-bit value cannot be accessed **atomically** on a 32-bit machine
- A spin-lock is used to synchronize access to `jiffies_64`; kernel routines call `get_jiffies_64()`
- But we don't want to have to increment two variables each tick
- Linker magic is used to make jiffies the low-order 32 bits of `jiffies_64`
- Not elegant!

4

Time of Day

- The time of day is stored in `xtime`, which is a `struct timespec`
- It's incremented once per tick
- Again, a spin-lock is used to synchronize access to it
- The apparent tick rate can be adjusted slightly, via the `adjtimex()` system call

Kernel Timers

- Two types of timers used by kernel routines
- Dynamic timer — call some routine after a particular interval
- Delay loops — tight spin loops for very short delays
- User-mode interval timers are similar to kernel dynamic timers

Dynamic Timers

- Specify an interval, a subroutine to call, and a parameter to pass to that subroutine
- Parameter used to differentiate different instantiations of the same timer — if you have 4 Ethernet cards creating dynamic timers, the parameter is typically the address of the per-card data structure
- Timers can be cancelled; there is (as usual) a delicate synchronization dance on multiprocessors

Spring 2026

Operating Systems

7

7

Delay Functions

- Spin in a tight loop for a short time — microseconds or nanoseconds
- Nothing else can use that CPU during that time, except via interrupt
- Used only when the overhead of creating a dynamic timer is too great for a very short delay

Spring 2026

Operating Systems

8

8

System Calls

- `time()` and `gettimeofday()`
- `adjtimex()` — tweaks apparent clock rate (even the best crystals drift)
- `setitimer()` and `alarm()` — interval timers for applications

Spring 2026

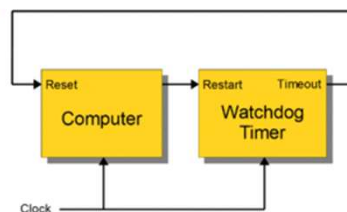
Operating Systems

9

9

Watchdog Timer

- Computer Operating Properly/COP timer...
- Used in embedded, remote automated systems where users cannot restart machines...



Spring 2026

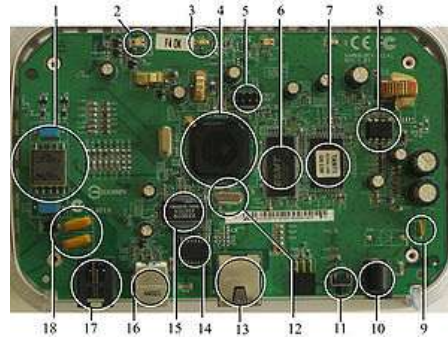
Operating Systems

10

10

What is an Embedded System

- A computer system with a dedicated function within a larger mechanical or electrical system, often with real-time computing constraints



ADSL modem/router. A modern example of an embedded system with a microprocessor (4), RAM (6), and flash memory (7)

Spring 2026

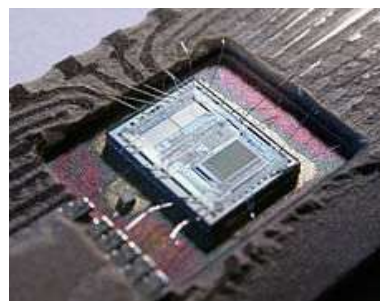
Operating Systems

11

11

What is a Microcontroller

- A small computer on a single integrated circuit containing a processor core, memory, and programmable input/output peripherals
- A microprocessor is a computer processor that incorporates the functions of a computer's central processing unit (CPU) on a single integrated circuit



Intel 8742: an 8-bit microcontroller that includes a CPU running at 12 MHz, 128 bytes of RAM, 2048 bytes of EPROM, and I/O in the same chip

Spring 2026

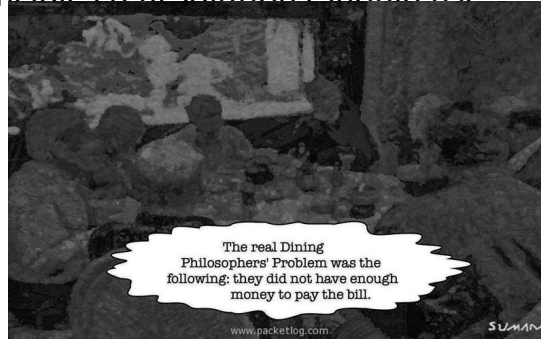
Operating Systems

12

12

Classic Synchronization Problems

- Invented to demonstrate synchronization primitives
- Not necessarily analogous to real-world problems
- Do illustrate principles real solutions should use



Spring 2026

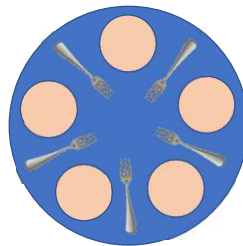
Operating Systems

13

13

The Dining Philosophers

- Five philosophers sitting around a table
- Each has a plate of pasta
- There are forks between each pair of plates
- Philosophers need **two** forks to eat



Spring 2026

Operating Systems

14

14

Thinking and Eating

- Philosophers spend their lives eating pasta and thinking
- When they're thinking, they don't need any implements
- To eat, they need to pick up the fork on their left and the fork on their right

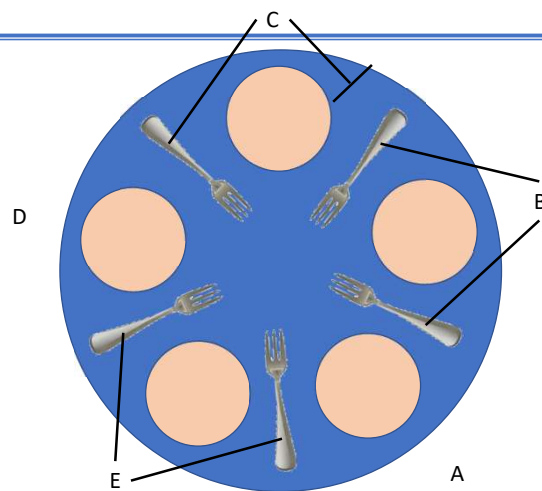
Spring 2026

Operating Systems

15

15

E, B and C Dining...



Spring 2026

Operating Systems

16

16

The Problem

- Find an algorithm by which they can all eat
- Must avoid deadlocks

```
void philosopher(int i) {  
    while(TRUE) {  
        think();  
        take_fork(i);  
        take_fork((i+1) % N);  
        eat();  
        put_fork(i);  
        put_fork((i+1) % N);  
    }  
}
```

A bad solution....

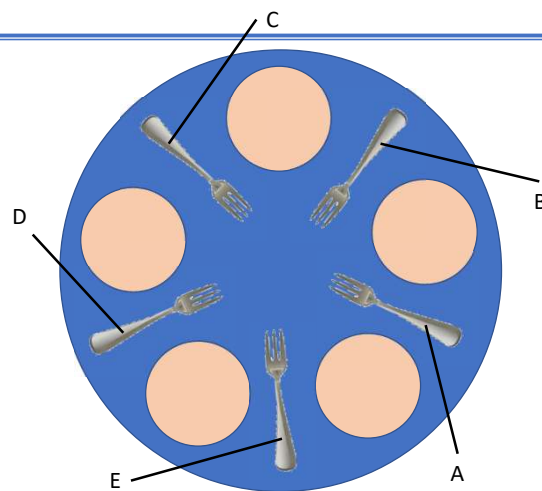
Spring 2026

Operating Systems

17

17

Everyone Grabs One Fork



Spring 2026

Operating Systems

18

18

Repairing the Solution

- Instead of take `fork((i+1) % N)`, see if it's available first
 - check and the grab are done inside a critical region
- If unavailable, sleep for a while and retry
- Still no good — what if everyone tries the left fork, waits, retries simultaneously, etc.
- Everyone can run, but no one makes progress: starvation

Spring 2026

Operating Systems

19

19

Randomization

- Suppose the philosophers sleep for a random time if they fail to get the second fork
- Usually, that will work fine
- Some applications require a guaranteed solution

Spring 2026

Operating Systems

20

20

A Related Quote from Knuth

“Finally, we need a great deal of faith in probability theory when we use hashing methods, since they are efficient only on the average, while their worst case is terrible! . . . Therefore hash tables are inappropriate for certain real-time applications such as air traffic control, where people’s lives are at stake.”

A Simple Guaranteed Solution

- Before taking a fork, a philosopher locks a mutex
- The philosopher can then take two forks, with no interference
- When done eating, the forks are replaced and the mutex is released

What's Wrong?

- As presented, only one philosopher can eat at a time
- Let's modify the algorithm: suppose that the philosopher — say, E — releases the mutex before eating
- What happens now? Does this solve the problem?

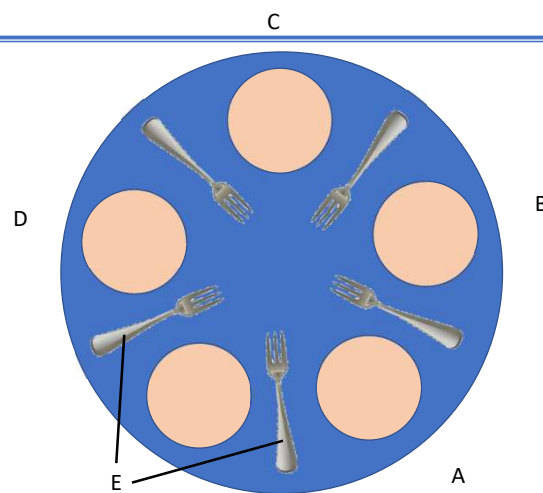
Spring 2026

Operating Systems

23

23

Everyone Grabs One Fork



Spring 2026

Operating Systems

24

24

No

- A, B, C, and D are now contending for the other three forks
- We're back to the same problem: how do we solve the problem fairly?

Spring 2026

Operating Systems

25

25

Principles of a Good Solution

- Won't go over the code here in detail
- Minor point: lock a mutex while trying to grab forks
- If they're both available, eat
- If one isn't available, mark yourself hungry and sleep
- When replacing forks, check if neighbors are hungry; if so, wake them

Spring 2026

Operating Systems

26

26

Why This Works: A Sketch of a Proof

- Suppose E is eating
- A and D can't eat; at most one of B and C can eat
- Assume B and C are thinking. Both A and D will wake up when E finishes, so things will obviously progress. Therefore, assume that C is eating, too.
- Everyone else who wants to eat is blocked
- When E finishes, A and D are awakened. D is still blocked, but A can run
- When C finishes, B can run
- Note: must make assumption that eating is a finite activity, and that there is a queue on each semaphore

Spring 2026

Operating Systems

27

27

Readers and Writers

- Suppose we have a database
- Any number of processes can read from it simultaneously; only one process can write to it at once
- No one can read while writing is taking place

Spring 2026

Operating Systems

28

28

Solution Sketch

- Keep track of how many readers there are (use a mutex)
- When the first reader comes along, lock the database mutex
- For subsequent readers, just count them
- When all readers are gone, release the database mutex
- When a writer comes along, lock the database mutex
- The write will block until all readers are done
- Danger: what if readers keep coming forever?

Spring 2026

Operating Systems

29

29

Lessons Learned

- Simple mutexes are not enough; we often need more than one
- Complex algorithms are hard to write and hard to understand — are they correct?
- Define how much “fairness” and “efficiency” you need — things like starvation and indefinite overtaking may be the real problems
- Watch for deadlocks

Spring 2026

Operating Systems

30

30

Deadlock

- Alice has taken the scanner, and wants to use the printer
- Bob has taken the printer, and wants to use the scanner
- Neither can progress until the other does...

Spring 2026

Operating Systems

31

31

History

- Studied a lot in the early days of operating systems
- Beloved by theorists
- Today, less an OS issue than an application issue, especially with databases
- Techniques are generally applicable

Spring 2026

Operating Systems

32

32

Resources

- Physical devices
- Database records
- Sharable things like main memory
- Highly system-dependent

Spring 2026

Operating Systems

33

33

Preemption

- Sometimes, an allocation can be preempted
- (If Alice outranks Bob, she can take the printer from him; he'll get both devices when she's done)
- Deadlocks don't occur if resources can be preempted
- Whether or not preemption can occur depends on the nature of the resource — a printer isn't preemptable in the middle of a job; main memory is if there's a swap area on disk

Spring 2026

Operating Systems

34

34

Sequence of Operations

- Request a resource; loop or block on failure
- Use it
- Release it
- The danger comes with two or more resources

Spring 2026

Operating Systems

35

35

```

process_A() {
    down(&resource_1);
    down(&resource_2);
    use_resources();
    up(&resource_1);
    up(&resource_2);
}
process_B() {
    down(&resource_1);
    down(&resource_2);
    use_resources();
    up(&resource_1);
    up(&resource_2);
}

```

```

process_A() {
    down(&resource_1);
    down(&resource_2);
    use_resources();
    up(&resource_1);
    up(&resource_2);
}
process_B() {
    down(&resource_2);
    down(&resource_1);
    use_resources();
    up(&resource_1);
    up(&resource_2);
}

```

Spring 2026

Operating Systems

36

36

Conditions for Deadlock (Coffman, 1971)

1. Mutual exclusion: Each resource is either available or assigned to exactly one process
2. Hold and wait: Processes holding resources can ask for more
3. No preemption: Resources cannot be taken away from a process involuntarily
4. Circular wait: There must be a chain of two or more processes waiting for the next member of the chain

If any of these conditions aren't present, deadlocks can't occur. Note that these conditions are tied to operating system policy.

Spring 2026

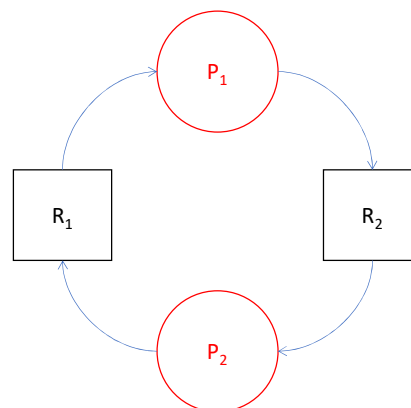
Operating Systems

37

37

Modeling a Deadlock with Directed Graphs

- An arc from a resource to a process means the resource is allocated to the process
- An arc from a process to a resource means the process is blocked waiting for the resource



Spring 2026

Operating Systems

38

38

Avoiding Deadlock

- An operating system can use this to watch for, and avoid, deadlock
- When a resource request occurs, see if granting it will create a cycle
- If so, delay granting that resource; run another process instead
- Wait for some other resources to be released — arcs to be cut — before granting the resource

Spring 2026

Operating Systems

39

39

Conditions for Deadlock (Coffman, 1971)

1. Mutual exclusion: Each resource is either available or assigned to exactly one process
2. Hold and wait: Processes holding resources can ask for more
3. No preemption: Resources cannot be taken away from a process involuntarily
4. Circular wait: There must be a chain of two or more processes waiting for the next member of the chain

If any of these conditions aren't present, deadlocks can't occur. Note that these conditions are tied to operating system policy.

Spring 2026

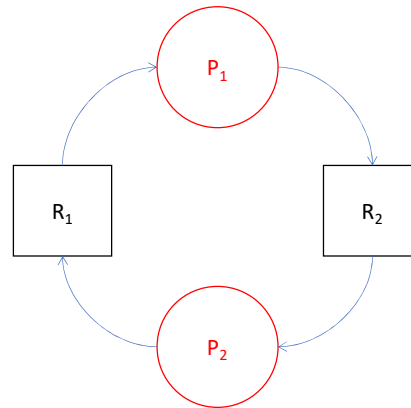
Operating Systems

40

40

Modeling a Deadlock with Directed Graphs

- An arc from a resource to a process means the resource is allocated to the process
- An arc from a process to a resource means the process is blocked waiting for the resource



Spring 2026

Operating Systems

41

41

Avoiding Deadlock

- An operating system can use this to watch for, and avoid, deadlock
- When a resource request occurs, see if granting it will create a cycle
- If so, delay granting that resource; run another process instead
- Wait for some other resources to be released — arcs to be cut — before granting the resource

Spring 2026

Operating Systems

42

42

General Philosophies

- Ignore the problem — maybe it isn't serious enough
- Detect it and recover
- Avoid it by careful resource allocation
- Prevent it by avoiding one of the four conditions

Spring 2026

Operating Systems

43

43

Ignoring Deadlocks

- Perhaps the chances of a deadlock are low
- Example: my desktop can have 532 processes running, and 1772 open files; each process can open 256 files
- Suppose every process that wanted to open a file and couldn't would sleep for a while and retry
- Easy to see that a deadlock could occur if lots of processes open lots of files
- Realistically, it just doesn't happen
- The problem is avoidable, but only at high cost

Spring 2026

Operating Systems

44

44

Detection and Recovery

- Simple case: one resource of each type
- Construct the digraph discussed earlier
- See if any cycles exist in the graph (algorithms are well-known)

Spring 2026

Operating Systems

45

45

Complex Case: Resource Classes

- There is more than one of certain resources (i.e., several printers)
- Let E_j be the total number of resource j ; let A_j be the number unallocated
- Create two matrices, C (current allocations) and R (requests). In each, row i corresponds to process i
- Invariant over n processes:

$$\forall j, \sum_{i=1}^n C_{ij} + A_j = E_j$$

- That is, all instances of resource j plus the number of j allocated is the total
- Vector comparison: $A \leq B$ iff $\forall j, A_j \leq B_j$

Spring 2026

Operating Systems

46

46

Algorithm

Start with all processes “unmarked”. Marked processes are able to run to completion.

1. Find an unmarked process P_i for which $R_i \leq A$ (i.e., a process whose resource requests can be satisfied now)
2. If found, let $A + C_i \rightarrow A$ (when this process finishes, its resource will become available); go back to step 1
3. If there are no such processes, the algorithm terminates

When the algorithm terminates, all unmarked processes are deadlocked

Spring 2026

Operating Systems

47

47

Example

A system has 4 tape drives, 2 plotters, 3 scanners, 1 CD-ROM

$$E = (TD \ P \ S \ CD)$$

$$E = (4 \ 2 \ 3 \ 1)$$

Process 1 has one scanner; process 2 has two tapes drives and a CD-ROM; process 3 has a plotter and two scanners

$$A = (2 \ 1 \ 0 \ 0)$$

$$C = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 2 & 0 & 0 & 1 \\ 0 & 1 & 2 & 0 \end{bmatrix}$$

Each process wants more

$$R = \begin{bmatrix} 2 & 0 & 0 & 1 \\ 1 & 0 & 1 & 0 \\ 2 & 1 & 0 & 0 \end{bmatrix}$$

Spring 2026

Operating Systems

48

48

Run the Algorithm

- Process 1 isn't runnable; no CD-ROMs are available
- Process 2 can't run; no scanners are available
- Process 3 is runnable; when it's done, $A = (2 \ 2 \ 2 \ 0)$
- That lets process 2 run; when it's done, $A = (4 \ 2 \ 2 \ 1)$
- That's enough to satisfy process 1, since

$$(2 \ 0 \ 0 \ 1) \leq (4 \ 2 \ 2 \ 1)$$

- No deadlock!

- Rerun the algorithm with $R = \begin{bmatrix} 2 & 0 & 0 & 1 \\ 1 & 0 & 1 & 1 \\ 2 & 1 & 0 & 0 \end{bmatrix}$
- $(1 \ 0 \ 1 \ 1) \not\leq (2 \ 2 \ 2 \ 0)$ — we have a deadlock

Spring 2026

Operating Systems

49

49

Recovery From Deadlock

- If possible, preempt a resource
- Figure out which resources are pre-emptible, and which process should be delayed
- Checkpoint state of process periodically. When deadlock occurs, roll some process back to before it acquired some resources
- Killing a process: find a process with resource, in the cycle or not, and kill it. Try to pick one that can be rerun without bad side-effects. . .

Spring 2026

Operating Systems

50

50

Deadlock Avoidance

- If we can detect deadlocks, can we avoid them?
- Yes, but. . .
- We can avoid deadlocks if certain information is available in advance

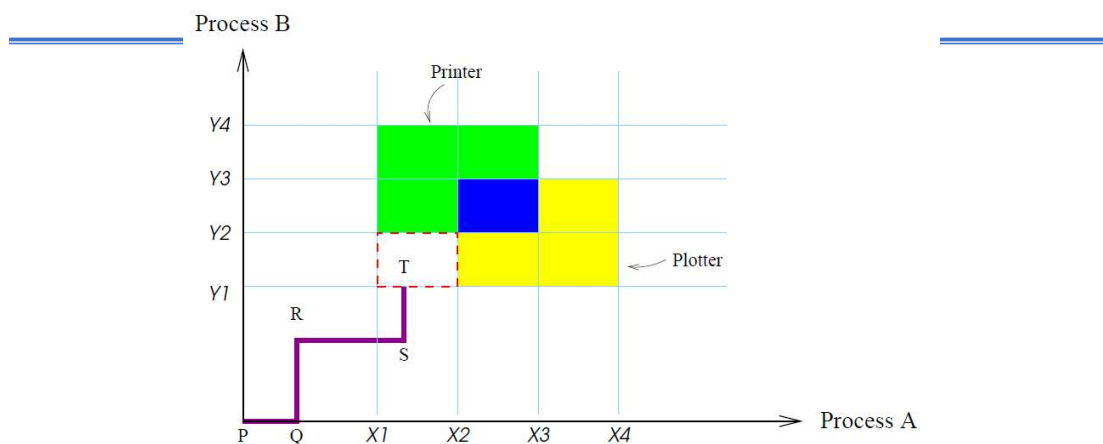
Spring 2026

Operating Systems

51

51

Process Trajectories



A needs the printer from X_1 to X_3 ; it needs the plotter from X_2 to X_4 .
 B needs the plotter from Y_1 to Y_3 ; it needs the printer from Y_2 to Y_4 .

Spring 2026

Operating Systems

52

52

Problems

- Warning sign: A and B are asking for resources in a different order
- Green region: both A and B have the printer — impossible
- Yellow region: both have the plotter
- Blue — both have both devices. . .
- The colored regions represent impossible states and cannot be entered

Spring 2026

Operating Systems

53

53

Avoiding Deadlock

- If the system ever enters the red-bordered state, it will deadlock
- At time t , cannot schedule B
- If we do, system will enter deadlock state
- Must run A until X4

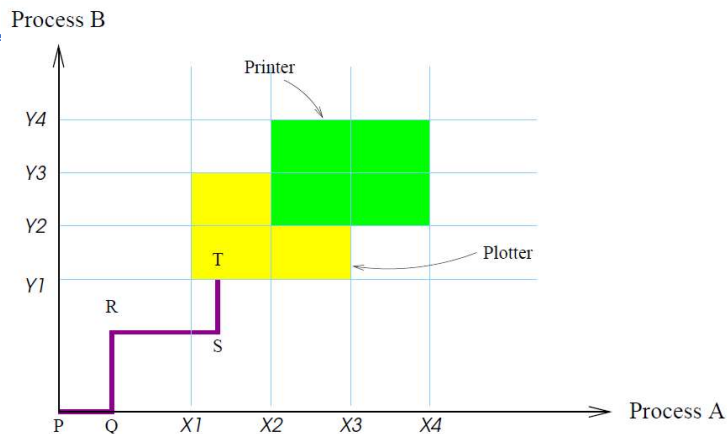
Spring 2026

Operating Systems

54

54

Alternate Process Trajectory



If A and B ask for the plotter first, there's no danger. B will clearly block if it's scheduled, so A will proceed. The dangerous state was where a process entered a clear box that would deadlock.

Spring 2026

Operating Systems

55

55

Safe and Unsafe States

- A state is safe if not deadlocked and there is a scheduling order in which all processes can complete, even if they all ask for all of their resources at once
- An unsafe state is not deadlocked, but no such scheduling order exists
- It may even work, if a process releases resources at the right time

Spring 2026

Operating Systems

56

56

The Banker's Algorithm (Dijkstra, 1965)

- Assume we're dealing with a single resource — perhaps dollars
- Every customer has a "line of credit" — a maximum possible resource allocation
- The banker only has a certain amount of cash on hand
- Not everyone will need all of their credit at once
- Solution: only grant requests if they leave us in a safe state

Spring 2026

Operating Systems

57

57

Example

	Has	Max		Has	Max		Has	Max
A	0	6	A	1	6	A	1	6
B	0	5	B	1	5	B	2	5
C	0	4	C	2	4	C	2	4
D	0	7	D	4	7	D	4	7
Free: 10			Free: 2			Free: 1		

The first state is safe; we can grant the requests sequentially. The second state is safe; we can grant C's maximum request, let it run to completion, and then have \$4 to give to B or D. If, after the second state, we give \$1 to B, we enter an unsafe state — there isn't enough money left to satisfy all possible requests.

Spring 2026

Operating Systems

58

58

The Banker's Algorithm for Multiple Resources

- Build matrices C (currently assigned) and R (and still needed), just as we used for deadlock detection
- Build vectors E (existing resources) and A (available), again as before
- To see if a state is safe:
 1. Find a row R whose unmet needs are $\leq A$
 2. Mark that row; add its resources to A
 3. Repeat until either all rows are marked, in which case the state is safe, or some are unmarked, in which case it's unsafe
- Run this algorithm any time a resource request is made

Spring 2026

Operating Systems

59

59

Why Isn't This Useful?

- Every process must state its resource requirements at startup
- This is rarely possible today
- Processes come and go
- Resources vanish as hardware breaks
- Not really used these days. . .

Spring 2026

Operating Systems

60

60

Example: File Binding Time

- Old mainframes: all file name binding is done immediately prior to execution.
- Also makes it easy to move files around
- Classic Unix: file names on command line (but not clearly identifiable as such) or compiled-in to commands. Occasional overrides via environment variables or options.
- GUIs: many files selected via menus

Early versus late binding is a major issue in system design
Both choices here have their advantages and disadvantages

Preventing Deadlocks

- Practically speaking, we can't avoid deadlocks
- Can we prevent them in the real world?
- Let's go back to the four conditions:
 1. Mutual exclusion
 2. Hold and Wait
 3. No preemption
 4. Circular wait

Attacking Mutual Exclusion

- Much less of an issue today — fewer single-user resources
- Many of the existing ones are dedicated to single machines used by single individuals, i.e., CD drives
- Printers are generally spooled
 - No contention; only the printer daemon requests it
- Not a general solution, but useful nevertheless

Spring 2026

Operating Systems

63

63

Attacking Holding and Wait

- Could require processes to state their requirements up front
- Still done sometimes in the mainframe world
- Of course, if we could do that, we could use the Banker's Algorithm

Spring 2026

Operating Systems

64

64

A Variant is Useful

- Before requesting a resource, release all currently-held resources
- Request all new ones at once
- Doesn't work if some resources must be held

Spring 2026

Operating Systems

65

65

Attacking Circular Wait

- Number each possible resource:
 1. Scanner
 2. Printer
 3. Tape drive
 4. ...
- Resources must be requested in numerical order
- Can't deadlock — prevents the out-of-order scenario we saw earlier
- Used on old mainframes
- Can combine this with release-and-rerequest

Spring 2026

Operating Systems

66

66

Mainframe Resources

- Wait until enough tape drives are available
- Wait until memory region is available
- Wait for all disk files to be free

Order based on typical wait time — disk files freed up quickly, while tape drives waited for operators to find and mount tape reels

Spring 2026

Operating Systems

67

67

Two-Phase Locking

- Frequently used in databases
- Processes need to lock several records then update them all
- Phase 1: try locking each record, one at a time
- On failure, release them all and restart
- When they're all locked, do the updates and then the release
- Effectively the same as “request everything up front”

Spring 2026

Operating Systems

68

68

What's Really Done About Deadlocks?

- In the OS, nothing. . .
- Overprovision some resources, such as process slots
- But — still very important in some applications, notably databases

Spring 2026

Operating Systems

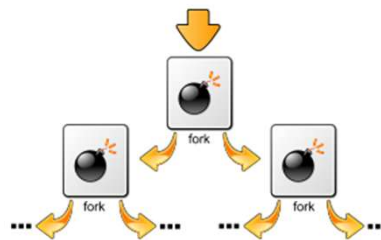
69

69

Fork Bomb

- A fork bomb (also called rabbit virus or wabbit) is a denial-of-service attack wherein a process continually replicates itself to deplete available system resources (e.g., process table fully saturated), causing resource starvation and slowing or crashing the system

```
import os
while True:
    os.fork()
```



Spring 2026

Operating Systems

70

70

What About Linux?

- No deadlock prevention or detection for applications or threads
- The kernel does care about deadlocks for itself

```
/* We can't just spew out the rules
 * here because we might fill the
 * available socket buffer space and
 * deadlock waiting for auditctl to
 * read from it... which isn't ever
 * going to happen if we're actually
 * running in the context of auditctl
 * trying to _send_ the stuff */
```

Spring 2026

Operating Systems

71

71

See you next time!

72